Getting Started with CECI High-Performance Computing

A Practical Guide

Guillaume Deside

April 9, 2026

High-Performance Computing for Research

Outline

1. Introduction to CECI
2. Getting Access & Connecting
3. Essential Linux Commands
4. SLURM Job Management
5. Practical Example: ML gridsearch
6. GPU Example: Deep Learning
7. R simple example: Parallel
8. R Project Organization on HPC

What is CECI?

Consortium des Équipements de Calcul Intensif

A Belgian Federation for HPC

- Federation of Belgian university HPC centers
- Shared computational infrastructure
- Available to **all Belgian researchers**
- 5 major clusters across Belgium
- Combined: **12,688 cores + 56 GPUs**
- Coordinated access and support

Why Use HPC?

- **Massive parallelization**
  Thousands of cores
- **Large memory capacity**
  Up to 2 TB per node
- **Long-running computations**
  Up to 21 days
- **GPU acceleration**
  Latest RTX/A-series
- **Big data storage**
  Up to 520 TB
- **Specialized software**
  Pre-installed modules

CECI Clusters Overview

Cluster	University	CPU	Cores	Memory	Network	Storage	Max Time
Lyra	ULB	Genoa 3.25 GHz	1,280	128 GB	50 GbE	CephFS 500 TB	5 days
Lemaitre4	UCLouvain	Genoa 2.4 GHz	5,120	766 GB	HDR Ib	BeeGFS 320 TB	2 days
NIC5	ULiège	Rome 2.9 GHz	4,672	256 GB-1 TB	HDR Ib	BeeGFS 520 TB	2 days
Hercules2	UNamur	Naples 2 GHz	1,024	256 GB-2 TB	10 GbE	NFS 80 TB	15 days
Dragon2	UMons	SkyLake 2.6 GHz	592	192-384 GB	10 GbE	RAID0 3.3 TB	21 days

`https://www.ceci-hpc.be/clusters.html`

Key Infrastructure

- AMD Genoa/Rome/Naples/SkyLake
- HDR InfiniBand (MPI clusters)
- Large-scale storage (80-520 TB)
- Latest GPU architectures

Step 1: Create Your CECI Account

Account Creation

Visit: `https://login.ceci-hpc.be/init/`

What You'll Need:

- University email address
- Choose a passphrase
- Institutional affiliation

What You'll Receive:

- Confirmation email
- Your CECI username
- **Private SSH key**: `id_rsa.ceci`

Next Step

Download and securely store your `id_rsa.ceci` key file

Keep your private key secure and never share it!

Different procedures for different operating systems

Windows

MobaXterm
(Recommended)

- Free SSH client
- File transfer
- Session manager

Mac / Linux

Terminal + SSH
(Built-in)

- Native SSH support
- Configure `.ssh/config`
- Simple setup
- Command-line

VS Code

Remote SSH
(All platforms)

- Visual interface
- Integrated terminal
- File explorer
- Code editing

Choose the method that works best for your workflow!

Windows: MobaXterm Setup

Installation & Configuration:

1. Download MobaXterm from `https://mobaxterm.mobatek.net/`
2. Install the free Home Edition
3. Import your `id_rsa.ceci` key
4. Configure SSH session with gateway

Video Tutorial

Complete setup walkthrough:
`https://www.youtube.com/watch?v=o41r0mFaURU`

Key Settings

- **Gateway:**
  `gwceci.cism.ucl.ac.be`
- **Username:**
  Your CECI login
- **Private key:**
  `id_rsa.ceci`
- **Cluster:**
  `lemaitre4.cism.ucl.ac.be`

MobaXterm simplifies gateway + cluster access

Mac/Linux: SSH Setup (1/3)

1. Store Your Private Key

```
# Create .ssh directory if it doesn't exist
mkdir -p ~/.ssh
# Move the downloaded key to .ssh folder
mv ~/Downloads/id_rsa.ceci ~/.ssh/
# Set correct permissions (IMPORTANT!)
chmod 600 ~/.ssh/id_rsa.ceci
```

For WSL (Windows Subsystem for Linux)

Your key might be at:
`/mnt/c/Users/<yourUsername>/Downloads/id_rsa.ceci`

Permission Error?

If SSH refuses the key with `"Permissions 0644 are too open"`
→ Make sure you ran `chmod 600`

Mac/Linux: SSH Setup (2/3)

2. Create Your Public Key

```
# Generate public key from private key
ssh-keygen -yf ~/.ssh/id_rsa.ceci > ~/.ssh/id_rsa.ceci.pub
```

You'll be asked for the passphrase you chose during account creation.

Error: "invalid format" or "error in libcrypto"?

Run this command to fix line endings:

```
dos2unix ~/.ssh/id_rsa.ceci
```

If `dos2unix` is not installed:

- Mac: `brew install dos2unix`
- Linux: `sudo apt install dos2unix`

Mac/Linux: SSH Setup (3/3)

3. Configure SSH Config File

Use the SSH Config Wizard

`https://www.ceci-hpc.be/sshconfig.html`

1. Choose your university
2. Provide your CECI username
3. Copy the generated configuration

4. Edit your config file:

```
# Open/create config file
nano ~/.ssh/config

# Paste the wizard output, then save (Ctrl+X, Y, Enter)

# Set correct permissions
chmod 600 ~/.ssh/config
```

Connecting to CECI Clusters

After configuration, connecting is simple:

```
# Connect to any cluster using the alias
ssh <cecicluster>

# Examples:
ssh lyra            # ULB
ssh lemaitre4       # UCLouvain
ssh nic5            # ULiege
ssh hercules        # UNamur
ssh dragon2         # UMons
```

SSH Gateway

UCLouvain users: Use `gwceci.cism.ucl.ac.be` as SSH gateway
See CISM documentation for details

VS Code Remote SSH

Setup:

1. Install VS Code
2. Install "Remote - SSH" extension
3. Open Command Palette (F1)
4. Select "Remote-SSH: Connect to Host"
5. Choose your cluster alias (e.g., `lemaitre4`)

Benefits

- Edit files directly on cluster
- Integrated terminal
- File explorer
- Git integration

Requirements

- SSH config already set up
- `id_rsa.ceci` key configured
- VS Code installed locally

First Connection

VS Code will:

- Install server on cluster
- Take a few minutes
- Only happens once

Navigation & File System

Moving Around

```
# Print current directory
pwd
# Change directory
cd /path/to/directory
cd ~                        # Go to home directory
cd ..                       # Go up one level
# List files and directories
ls                          # Basic listing
ls -l                       # Detailed listing (permissions, size, date)
ls -lh                      # Human-readable sizes (KB, MB, GB)
```

Quick Tip

Use Tab for auto-completion of paths and filenames!

Creating & Managing Files/Directories

Creating:

```
# Create directory
mkdir myproject
mkdir -p path/to/dir
```

Moving/Renaming:

```
# Move or rename
mv oldname.txt newname.txt
mv file.txt /path/to/dir/
```

Copying:

```
# Copy file
cp file.txt backup.txt
# Copy directory
cp -r myproject/ backup/
```

Deleting:

```
# Remove file
rm file.txt
# Remove directory
rm -r mydir/
```

Warning

`rm` is permanent! No trash/recycle bin!

File Permissions with chmod

Understanding Permissions:

```
ls -l myfile.txt
# Output: -rwxr-xr-- 1 user group 1234 Jan 01 12:00 myfile.txt
#         -rwxr-xr--
#          |||||||||
#          ||||||++-- Others: r-- (read only)
#          |||+++---- Group:  r-x (read, execute)
#          +++------- Owner:  rwx (read, write, execute)
```

Numeric Method:

```
chmod 600 id_rsa.ceci
# Owner: 6 (rw-)
# Group: 0 (---)
# Other: 0 (---)
```

Symbolic Method:

```
chmod +x script.sh
```

Disk Usage & Storage Management

Check Disk Space:

```
# Show disk usage
df -h
# Check your quota
quota -s
```

File Compression:

```
# Compress file
gzip largefile.txt
# Creates: largefile.txt.gz
# Decompress
gunzip largefile.txt.gz
# Create tar archive
tar -czf archive.tar.gz mydir/
# Extract tar archive
tar -xzf archive.tar.gz
```

HPC Best Practice

Compress large data files to save quota:
`gzip results.csv` can reduce size by 80-90%!

Understanding SLURM

What is SLURM?

- **S**imple **L**inux **U**tility for **R**esource **M**anagement
- Job scheduler used on CECI clusters
- Manages compute resources
- Ensures fair access for all users
- Queues and prioritizes jobs

Key SLURM Commands

`sinfo`	Cluster status
`sbatch`	Submit job
`squeue`	View queue
`scancel`	Cancel job
`sacct`	Job history
`srun`	Interactive job

Remember

Never run computationally intensive jobs on login nodes! Always use SLURM.

sinfo - Cluster Information

Check cluster status and available resources:

```
# Basic cluster information
sinfo
# Detailed view
sinfo -l
# Show only available nodes
sinfo -t idle
# Node-specific information
sinfo -N
```

Example Output:

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
Def          up   2-00:00:00     40  alloc node[001-040]
Def          up   2-00:00:00     88   idle node[041-128]
gpu          up   5-00:00:00      2  alloc gpu[01-02]
gpu          up   5-00:00:00     38   idle gpu[03-40]
```

Understanding the Output

sbatch - Submitting Batch Jobs

Submit a job script to the queue:

```
# Basic submission
sbatch myjob.sbatch
```

Example Job Script (`myjob.sbatch`):

```
#!/bin/bash
#SBATCH --job-name=my_analysis
#SBATCH --output=output_%j.log
#SBATCH --error=error_%j.log
#SBATCH --time=02:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem="16G"

# Run your code
python3 analysis.py
```

sbatch - Common SBATCH Options

Option	Description
`-job-name=NAME`	Job name (appears in queue)
`-output=FILE`	Standard output file (%j = job ID)
`-error=FILE`	Standard error file
`-time=HH:MM:SS`	Maximum runtime (e.g., 02:30:00)
`-ntasks=N`	Number of tasks (for MPI)
`-cpus-per-task=N`	CPUs per task
`-mem=SIZE`	Memory (e.g., 16G, 32GB)
`-mem-per-cpu=SIZE`	Memory per CPU
`-partition=NAME`	Partition/queue (gpu, Def, etc.)
`-gres=gpu:N`	Request N GPUs
`-array=1-10`	Job array (10 similar jobs)
`-mail-user=EMAIL`	Email for notifications
`-mail-type=TYPE`	When to email (BEGIN, END, FAIL, ALL)

squeue - View Job Queue

Check status of jobs in the queue:

```
# View only YOUR jobs
squeue --me
# View all jobs
squeue
# Detailed view
squeue -l
```

Example Output:

```
JOBID     NAME          USER      ST   TIME   NODES NODELIST
123456    my_analysis   jdoe      R    1:23   1     node042
123457    data_proc     jdoe      PD   0:00   1     (Priority)
```

- `R` = Running, `PD` = Pending, `CG` = Completing
- `TIME` = How long job has been running

squeue - Understanding Job States

State	Code	Meaning
Running	R	Job is currently running
Pending	PD	Job is waiting in queue
Completing	CG	Job is finishing up
Completed	CD	Job finished successfully
Failed	F	Job failed
Cancelled	CA	Job was cancelled
Timeout	TO	Job exceeded time limit
Node Fail	NF	Node failed during job
Out of Memory	OOM	Job exceeded memory limit

scancel - Cancel Jobs

Cancel running or pending jobs:

```
# Cancel specific job
scancel 123456
# Cancel all YOUR jobs
scancel -u $USER
# Cancel by job name
scancel --name=my_analysis
# Cancel jobs in specific state
scancel -u $USER --state=PENDING
# Cancel job array
scancel 123456_[1-10]
# Cancel specific array task
scancel 123456_5
```

sacct - Job Accounting & History

View completed and running job information:

```
# View your recent jobs
sacct
# View specific job
sacct -j 123456
# Jobs from today
sacct --starttime=today
# All jobs with specific name
sacct --name=my_analysis
# View failed jobs only
sacct --state=FAILED
```

Quick Reference: SLURM Commands

Command	Purpose
`sinfo`	View cluster status and partitions
`sinfo -N`	View node-level information
`sbatch script.sh`	Submit batch job
`squeue -me`	View YOUR jobs in queue
`squeue -u $USER`	View YOUR jobs (alternative)
`squeue`	View ALL jobs in queue
`squeue -j 123456`	View specific job status
`scancel 123456`	Cancel specific job
`scancel -u $USER`	Cancel ALL your jobs
`sacct`	View job history
`sacct -j 123456`	View specific job details
`sacct -S today`	View today's jobs

Master these commands for efficient HPC workflow!

Example 1: Python with Built-in Parallelization

Scenario: Your Python code uses functions that parallelize internally (e.g., scikit-learn's `GridSearchCV` with `n_jobs=-1`)

ONE-TIME SETUP: Create Virtual Environment

```
# Create virtual environment
python3 -m venv env
# Activate environment
source env/bin/activate
pip install numpy pandas matplotlib
pip install -U scikit-learn
```

EVERY SESSION: Activate Environment

```
source env/bin/activate
```

Before running any Python code, always activate your environment first!

Example: Machine Learning with Scikit-Learn

Scenario

Train a Random Forest model on a large dataset

- Dataset: 10,000 samples, 20 features
- Model: Random Forest with 1000 trees
- Task: Hyperparameter tuning with Grid Search

Step 1: Python Script

```python
rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(
    rf, param_grid, cv=5,
    n_jobs=-1,  # Use all available cores!
    verbose=2
)
```

Two Ways to Submit Jobs

Method 1: Manual SBATCH Script

- Create `.sbatch` file with SBATCH directives
- Submit with `sbatch script.sbatch`
- Good for: simple, one-time jobs
- Easy to understand and modify

Best for:

- Learning SLURM
- Quick experiments

Method 2: Python-Generated Scripts

- Python function creates SBATCH script
- Automatically submits job
- Good for: parameter sweeps, many jobs
- Programmatic job management

Best for:

- Batch experiments
- Automated workflows

Choose the method that fits your workflow!

Method 1: Manual SBATCH Script

Create a file: `example_cpu.sbatch`

```
#!/bin/bash
#SBATCH --job-name=Example_CPU
#SBATCH --time=00:30:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=25
#SBATCH --mem-per-cpu=400
#SBATCH --mail-user=guillaume.deside@uclouvain.be
#SBATCH --mail-type=FAIL
#SBATCH --mail-type=END
#SBATCH --output=example_CPU.txt

python3 example_ml.py
```

Submit the job:

```
sbatch example_cpu.sbatch
# Output: Submitted batch job 123456
```

Method 2: Using the Python Submission Function

Example: Submit CPU Job (python3 submit.py)

```python
# Define job configuration
job_info = {
    "job_name": "Example_CPU",
    "time": "00:30:00",
    "ntasks": 1,
    "cpus_per_task": 25,
    "mem_per_cpu": 400,  # MB
    "mail_user": "guillaume.deside@uclouvain.be",
    "output": "example_CPU.txt",
    "script": "python3 example_ml.py"
}
# Submit the job
job_id = submit_job(job_info, "example_cpu.sbatch")
print(f"Submitted job {job_id}")
```

Real Example: Random Forest Results

Job submitted with 25 CPUs:

```
[INFO] CPU Detection:
  - Total CPUs available: 128
  - SLURM allocated CPUs: 25
  - Using SLURM allocation

[STEP 1/4] Generating synthetic dataset...
[STEP 3/4] Starting Grid Search...
Fitting 5 folds for each of 8 candidates, totalling 40 fits
[STEP 4/4] Training Results:
  - Total training time: 6.56 minutes (393.9 seconds)
  - Best cross-validation score: 0.9545
  - Best parameters:
      max_depth: 20
      min_samples_split: 5
      n_estimators: 500
[PERFORMANCE METRICS]
  - Number of CPUs used: 128
  - SLURM allocation: 25 CPUs
  - Wall-clock time: 393.9 seconds
```

Understanding HPC Benefits

Important Note About Performance

Modern laptops (especially M-series Macs) may have **faster CPUs** than older HPC nodes, making direct speed comparisons misleading.

Real Benefits of HPC (Beyond Raw Speed)

1. **Free Your Computer**
   - Submit job and continue working
   - No laptop battery drain
   - No overheating concerns

Real Benefits of HPC (Beyond Raw Speed)

Real Benefits of HPC (Beyond Raw Speed) - continued

2. **Scale Beyond Your Hardware**
   - Access 25-128 CPUs (vs your 4-8 cores)
   - Use up to 2TB RAM (vs your 16-32GB)
   - Access professional GPUs
3. **Run Multiple Experiments**
   - Submit 10 jobs simultaneously
   - Parameter sweeps across many configs
   - Overnight runs without worry

Laptop vs HPC: The Real Comparison

Your Laptop

Advantages:

- Newer, faster CPU
- Immediate access
- No queue time
- Easy debugging

Limitations:

- Computer unavailable
- Limited cores (4-12)
- Limited RAM (8-32GB)
- Battery/heat issues
- One job at a time

HPC Cluster

Advantages:

- Computer stays free
- Many cores (25-128)
- Huge RAM (up to 2TB)
- No hardware concerns
- Multiple jobs in parallel
- Professional GPUs

Limitations:

- Older CPUs
- Queue waiting time
- Remote access needed

Practical Workflow: Parameter Sweep

Use Python to submit multiple jobs with different parameters:

```python
cpu_configs = [1, 4, 8, 16, 32]

for n_cpus in cpu_configs:
    job_info = {
    ... (some parameters)
        "ntasks": 1,
        "cpus_per_task": n_cpus,
        "mem_per_cpu": 400,
        "output": f"rf_cpu_{n_cpus}_%j.log",
        "script": f"python3 train_rf.py --n_jobs={n_cpus}"
    }
    job_id = submit_job(job_info, f"rf_cpu_{n_cpus}.sbatch")
    print(f"Submitted job {job_id} with {n_cpus} CPUs")
```

Benefit

Test 5 configurations simultaneously instead of sequentially!

When to Use Each Method

Scenario	Method	Why
Learning SLURM	Manual script	Easy to understand
Single experiment	Manual script	Quick and simple
Quick test	Manual script	Less overhead
Parameter sweep	Python function	Automate variations
Many similar jobs	Python function	Avoid repetition
Complex workflows	Python function	Programmatic control
Job dependencies	Python function	Manage job chains
Production pipeline	Python function	Reproducibility

Tip

Start with manual scripts to learn, then transition to Python automation for complex workflows!

Example: CNN Image Classification

Scenario

Train a Convolutional Neural Network on CIFAR-10 dataset

- Dataset: 50,000 training images (32×32 pixels, 10 classes)
- Model: Simple CNN with 3 convolutional layers
- Task: Train for 20 epochs

CPU (32 cores)

- Training time: **15 minutes**
- Limited by sequential ops
- High CPU usage

GPU (1× RTX 6000)

- Training time: **90 seconds**
- Optimized for matrix ops

Deep Learning = GPU's sweet spot!

Python Script

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import time
import sys

# Check if GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"GPU Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
```

GPU Jobs: Special Considerations

Important: GPU-Specific Setup

GPU jobs require CUDA-compatible packages and correct partition selection

Step 1: Load Python Module (Manneback Example)

```
# Find available Python versions
module spider Python

# Load specific Python version with GPU support
module load releases/2023a
module load Python/3.11.3-GCCcore-12.3.0

# Verify Python version
python3 --version
```

GPU Jobs: Special Considerations

Step 2: Create Virtual Environment & Install PyTorch

```
# Create and activate virtual environment
python3 -m venv env
source env/bin/activate

# Install PyTorch with CUDA 12.1 support
pip install torch torchvision --index-url https://download.pytorch.org/
    whl/cu121
```

GPU Jobs: Testing & Submission

Step 3: Test GPU Access Interactively

```
# Request interactive GPU node directly
srun --partition=gpu --gres=gpu:1 --cpus-per-task=4 --pty bash

# Verify GPU is available
nvidia-smi
```

Step 4: Submit GPU Job

```
# Submit your GPU job script
sbatch gpu_job.sbatch
```

Remember

Always specify `-partition=gpu` and `-gres=gpu:N` in your job script!

GPU Job Script Example

File: `gpu_job.sh`

```
#!/bin/bash
#SBATCH --job-name=Example_GPU
#SBATCH --partition=gpu          # Use GPU partition
#SBATCH --gres=gpu:1              # Request 1 GPU
#SBATCH --cpus-per-task=4         # 4 CPUs for data loading
#SBATCH --mem=16G                 # 16GB RAM
#SBATCH --time=00:30:00           # 30 minutes max
#SBATCH --output=example_GPU.txt  # Output file

# Activate virtual environment (if using one)
source ~/myproject/env/bin/activate

# Run your GPU-accelerated Python script
python3 example_GPU.py
```

GPU Job Output: Device Detection

Beginning of `example_GPU.txt`:

```
[INFO] Device Detection:
  - Device selected: cuda
  - GPU Name: NVIDIA A10
  - GPU Memory: 23.60 GB
  - CUDA Version: 12.1
  - Number of GPUs: 1
```

If You See "cpu" Instead

Check: PyTorch CUDA installation, `-partition=gpu`, `-gres=gpu:1`

GPU Job Output: Training Progress

Training output showing GPU speedup:

```
Epoch  1/20 (  4.9s) | Train Loss: 1.5012 Acc: 45.00% | Test Loss: 1.2427 Acc:
    55.35% *BEST*
============================================================
TRAINING COMPLETED
============================================================
Total training time: 1.57 minutes (94.2 seconds)
Average time per epoch: 4.7 seconds
Final test accuracy: 88.23%
Best test accuracy: 89.12%
Device used: cuda
============================================================
```

CPU vs GPU: Understanding the Difference

CPU (Central Processing Unit)

- General-purpose processor
- Few powerful cores (4-32 typically)
- Optimized for sequential tasks and complex logic

GPU (Graphics Processing Unit)

- Specialized processor originally for graphics
- Thousands of smaller cores (1000s-10000s)
- Excels at repetitive mathematical computations

Analogy:

CPU = 1 expert chef (fast, versatile, handles complex recipes)

GPU = 1000 line cooks (slower individually, but process massive quantities)

When to Use GPU vs CPU

Use GPU For:

- **Deep learning**
  CNNs, RNNs, Transformers
- **Large neural networks**
  Millions of parameters
- **Computer vision**
  Image/video processing
- **Natural language**
  BERT, GPT-style models
- **Large matrix ops**
  Linear algebra heavy

CPU is Better For:

- **Traditional ML**
  Random Forest, XGBoost, SVM
- **Statistical models**
  Linear/logistic regression
- **Small networks**
  Few layers, small datasets
- **Sequential tasks**
  Non-parallelizable code
- **Memory-bound**
  Large data processing

Installing R Packages on the Cluster

Important: Environment Setup First

Before installing packages, set these environment variables:

```
export LC_ALL=C
export LANG=C
```

This prevents locale-related errors during package installation.

Step 1: Load R Module

```
# Find available R versions
module spider R
# Load R module
module load R
# Verify R is loaded
R --version
```

Installing R Packages: Interactive Method

Install packages interactively

```
# Load R
module load R

# Start R
R
```

Inside R:

```
# Install required packages
install.packages("randomForest")  # Machine learning

# Verify installation
library(randomForest)

# Exit R
q()
```

Package Installation: Common Issues

Issue 1: "Non-zero exit status"

Solution: Install dependencies separately

```
# Install dependencies first
install.packages("Matrix")
# Then install the main package
```

Issue 2: "Permission denied"

Solution: Packages install to your home directory automatically
Check: `~/R/x86_64-pc-linux-gnu-library/`

Issue 3: "Package not available"

Solution: Check package name or try different repository

```
install.packages("PackageName", repos = "http://cran.r-project.org")
```

Example 2: Random Forest with Parallel Cross-Validation

Scenario: Machine learning model validation with built-in parallelization

What This Example Does

1. Generates synthetic classification dataset (50,000 samples, 30 features)
2. Trains Random Forest classifier
3. Performs 10-fold cross-validation, repeated 5 times (50 total fits)
4. Runs in parallel across multiple CPU cores
5. Reports accuracy statistics

The R Script: Key Components

File: `parallel_analysis.R`

1. CPU Detection

```r
num_cores <- detectCores()
slurm_cpus <- Sys.getenv("SLURM_CPUS_PER_TASK")
if (slurm_cpus != "") {
    num_cores <- as.integer(slurm_cpus)  # Use SLURM allocation
}
```

2. Parallel Backend Setup

```r
library(doParallel)
library(foreach)

cl <- makeCluster(num_cores)
registerDoParallel(cl)
```

The R Script: Parallel Execution

3. Parallel Cross-Validation Loop

```r
results <- foreach(i = 1:total_iterations,
                   .combine = rbind,
                   .packages = c("randomForest")) %dopar% {

    # Create random train/test split
    # Train Random Forest
    # Predict and evaluate
    # Return results

}
```

The SLURM Job Script

File: `submit_rf.sh`

```
#SBATCH --job-name=Example_R
#SBATCH --time=00:30:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=20        # Request 20 CPUs
#SBATCH --mem-per-cpu=4000        # 4GB per CPU = 80GB total
#SBATCH --output=example_R.txt

# Set environment
export LC_ALL=C
export LANG=C
# Load R module
module load R
# Run R script
Rscript parallel_analysis.R
```

The 4-File System for Parallel R

Why 4 Separate Files?

Separating concerns makes code:

- Easier to debug
- Reusable across projects
- Modular and maintainable
- Clear in workflow

The 4 Files:

1. `submit.sh` - SLURM configuration
2. `source.R` - Parameters & functions
3. `run.R` - Parallel execution
4. `out.R` - Results aggregation

Workflow:

1. Submit job with `submit.sh`
2. Each task runs `run.R`
3. `run.R` sources `source.R`
4. After all tasks: run `out.R`

Clean separation = Easy troubleshooting!

File 1: submit.sh - The Job Submission Script

```
#SBATCH --array=0-49            # Create 50 tasks (IDs: 0-49)
#SBATCH --mail-type=ALL         # Email when starts/ends
#SBATCH --mail-user=your.email@uclouvain.be
#SBATCH --job-name=MySim        # Name in queue
#SBATCH --output=Output_%a.Rout  # Each task logs to Output_0.Rout, etc.
#SBATCH --time=01:00:00         # 1 hour max per task
#SBATCH --mem-per-cpu=2000      # 2GB RAM per CPU
#SBATCH --cpus-per-task=4       # 4 CPUs per task

module load R                   # Load R module
Rscript ~/myproject/run.R       # Run this script for each task
```

Key Concepts

- -array=0-49: Creates 50 independent tasks
- Each task gets 4 CPUs $\rightarrow$ Total: 50 tasks $\times$ 4 CPUs = 200 CPUs used
- %a in filenames = array task ID

File 2: source.R - Parameters and Functions

Purpose: Define simulation parameters and the core function

```
# ============================================================
# FILE: source.R
# PURPOSE: Define all simulation parameters and functions
# ============================================================
# ---- 1. SIMULATION PARAMETERS ----
#....

# ---- 2. THE CORE FUNCTION ----
# This function runs ONE simulation iteration
run_one_simulation <- function(iter_id) {

                #...
}
```

source.R - Why Separate This File?

Advantages of source.R

- **Single source of truth**: All parameters in one place
- **Easy to modify**: Change `n_samples` once, affects all tasks
- **Reusable**: Same function across different analyses
- **Testable**: Can run `run_one_simulation(1)` locally to test
- **Readable**: Clear documentation of what simulation does

Testing Locally

Before submitting to cluster, test on your laptop:

- Source the file: `source("source.R")`
- Run one iteration: `result <- run_one_simulation(1)`
- Check output: `str(result)`
- Estimate runtime: `system.time(run_one_simulation(1))`

File 3: run.R - The Parallel Executor

```r
# ============================================================
# FILE: run.R
# PURPOSE: Run parallel simulations for one SLURM array task
# ============================================================

# ---- 1. LOAD PACKAGES ----
# ---- 2. LOAD PARAMETERS AND FUNCTION ----
# ---- 3. GET TASK INFORMATION FROM SLURM ----
# ---- 4. DETERMINE WHICH ITERATIONS THIS TASK HANDLES ----
# ---- 5. SET UP PARALLEL BACKEND ----
# ---- 6. RUN SIMULATIONS IN PARALLEL ----
# ---- 7. CLEAN UP ----
# ---- 8. SAVE RESULTS ----
```

run.R - Step-by-Step Explanation

1. **Load packages**: Get parallel computing tools
2. **Source parameters**: Load everything from `source.R`
3. **Get SLURM info**:
   - `SLURM_ARRAY_TASK_ID`: Which task am I? (0-49)
   - `SLURM_CPUS_PER_TASK`: How many CPUs do I have? (4)
4. **Calculate my work**:
   - Task 0: iterations 1-20
   - Task 1: iterations 21-40, etc.
5. **Create parallel cluster**: Use my 4 CPUs as workers
6. **Run simulations**: Distribute 20 iterations across 4 CPUs
7. **Save results**: Each task saves its own `.rds` file

Why foreach + doParallel?

- Automatically distributes work across CPUs
- Handles data collection with `.combine`
- Clean syntax for embarrassingly parallel tasks

File 4: out.R - Results Aggregation

```
# ============================================================
# FILE: out.R
# PURPOSE: Aggregate results from all tasks
# RUN THIS AFTER all tasks have completed
# ============================================================

# ---- 1. LOAD PARAMETERS ----


# ---- 2. COLLECT RESULTS FROM ALL TASKS ----

# ---- 3. COMPUTE STATISTICS ----

# ---- 4. SAVE AGGREGATED RESULTS ----
```

out.R - When and How to Run

IMPORTANT: Timing

Run `out.R` ONLY after ALL tasks have completed!

How to Run out.R

Option 1: Interactive (recommended)

- After receiving "job completed" email
- Run: `R CMD BATCH out.R &`

Option 2: Automated (advanced)

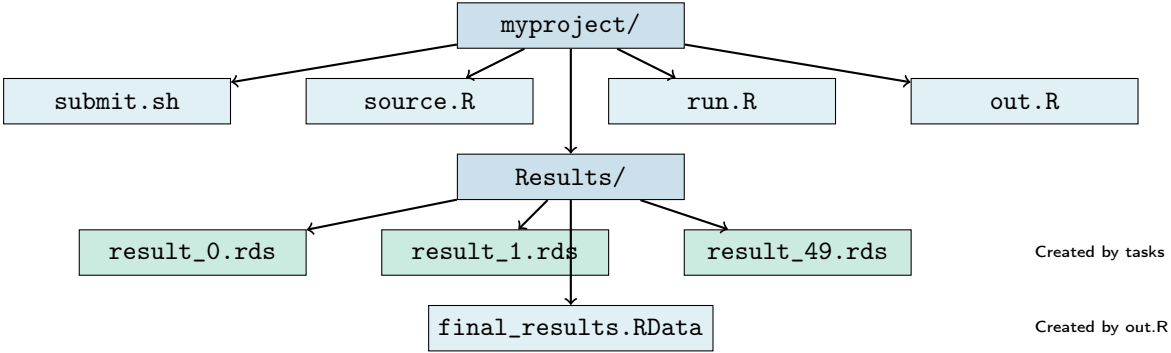
- Create separate SLURM job with `-dependency` flag
- Runs automatically after array job finishes

Verification

Check `out.Rout` file for any errors or missing tasks

Complete Directory Structure

Your project folder on the cluster:



Before Submitting

Create the Results folder: `mkdir Results`